Sensore di temperatura ed umidità



Versione 1.0

INDICE

1. Analisi **2**
1.1 Testo assegnato 2
1.2 Analisi del problema 3
1.2.1 Il Protocollo 1-Wire e il DS18B20 4
1.2.2 Il Comando Convert T [44h] e lo Scratchpad 6
1.2.3 Obiettivi e Contesto 9
1.2.3.1 Elementi Computazionali e Specifiche Numeriche 9
1.2.4 Flusso Logico (Flowchart Concettuale) 11
1.3 Analisi della soluzione adottata 13
1.3.1 Flowchart generale 13
1.3.2 Flowchart tecnico descrittivo del firmware 14
1.3.3 Flowchart ricezione comando inviato dal master e interpretazione 17
1.3.4 Flowchart funzione inizializzazione Timer 18
1.3.5 Flowchart funzione per la lettura dei dati rilevati dal sensore e successivo aggiornamento dello scratchpad (readDHT22) 20
1.3.6 Statechart complessivo 25
1.4 Analisi delle soluzioni alternative 27
2. Sintesi **28**
2.1 Sintesi componente ME1 28
2.2 Sintesi componente ME2 29
3. Sistema complessivo **31**
3.1 Manuale utilizzo 31
4. Bibliografia **33**

1.Analisi

1.1 Testo assegnato

Realizzare un sistema in grado di rilevare temperatura ed umidità da un sensore DHT22 https://docs.wokwi.com/parts/wokwi-dht22/ e sia in grado trasmettere i valori letti secondo il protocollo 1-Wire in analogia al sensore https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf
Il Sistema dovrà, in particolare:

- Definire il tracciato dello scratchpad da 64 bit in grado di contenere le informazioni relative alla temperatura ed umidità con la precisione di due cifre decimali;
- interpretare il comando “Convert” [44h] ed avviare una singola conversione di temperatura ed umidità. Dopo la conversione, il dato risultante viene memorizzato nella memoria scratchpad e quindi ritorna al suo stato di inattività a basso consumo.

La progettazione dovrà essere svolta secondo il paradigma:
• Minori risorse impiegate per la macchina
• Maggior sforzo per la progettazione e sviluppo.
In particolare:
a) componenti HW-SW dovranno essere "Automi" indipendenti che comunicano con variabili globali
b) Le cadenze temporali (ritardi ecc.) dovranno essere governati dall'ISR del clock
c) La progettazione delle ISR dovrà garantire tempi di servizio compatibili con le risposte al tempo reale del sistema.
d) La gestione degli I/O deve essere diretta, senza usare le funzioni di "Arduino" o librerie esterne.

1.2 Analisi del problema

Per arrivare a capire come muoversi per implementare una logica adeguata per simulare il protocollo One-Wire, in analogia al sensore **DS18B20** [2], è stata consultata per prima la documentazione di quest'ultimo e successivamente, ricercando informazioni sul sensore richiesto, si è evinto che, nonostante anche il **DHT22** invii dati utilizzando una singola linea, il formato non è compatibile con Dallas One-Wire.

Il sensore **DHT22** non può comunicare usando il protocollo **Dallas 1-Wire** perché utilizza un protocollo di comunicazione proprietario e completamente diverso, esso infatti utilizza un protocollo a livello di bit personalizzato che non è standardizzato come l'altro.

Struttura protocollo DHT22:

- La comunicazione avviene tramite un singolo filo dati (oltre a VCC e GND), ma il formato dei segnali è diverso.
- Il DHT22 richiede un segnale di **start** inviato dal microcontrollore, seguito da una sequenza di bit di dati trasmessi dal sensore.
- I dati vengono trasmessi sotto forma di sequenza di impulsi di diversa durata (24-26 µs per "0" e circa 70-71 µs per "1").

D'altro canto, si può vedere che il **Protocollo Dallas 1-Wire**:

- Supporta la comunicazione tra un *master* (es. microcontrollore) e più *slave* (es. sensori).
- Ogni dispositivo ha un **ID univoco a 64 bit**, che consente di gestire più sensori sulla stessa linea dati.
- I dati vengono trasmessi utilizzando slot temporizzati per rappresentare bit di "1" o "0".
- Utilizza un formato di comando e risposte più complesso rispetto al protocollo del DHT22.

Caratteristica	DHT22	Dallas 1-Wire
Protocollo	Proprietario	Standardizzato (Maxim Integrated)
Durata impulsi	Basata sulla lunghezza dell'impulso	Basata su slot temporizzati
Multi-dispositivo	Non supportato	Supportato (grazie all'ID univoco)
Hardware richiesto	Linea semplice	Linea dati con pull-up resistore
Formato dati	Dati temperatura e umidità diretti	Dati grezzi con comando e CRC

Tabella che mostra le differenze tecniche principali fra i due protocolli.

1.2.1 Il Protocollo 1-Wire e il DS18B20

Avendo raccolto queste informazioni, procedendo con l'analisi del problema, è sempre più evidente che la risposta alla richiesta sta, quindi, nel sfruttare la parte **firmware** per emulare il comportamento del protocollo adottato dal sensore **DS18B20**[2][5]:

Il DS18B20 utilizza il protocollo 1-Wire[3], il quale si basa su una singola linea di comunicazione (più una massa) e permette la comunicazione bidirezionale tra master e uno o più dispositivi slave. Di seguito vengono evidenziati gli aspetti principali.

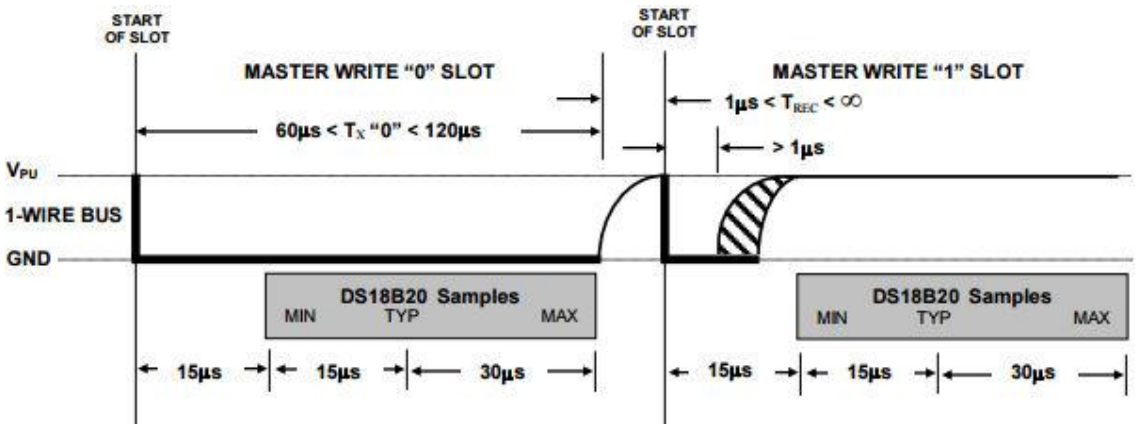
Inizializzazione e Reset

1. Reset da parte del master:
2. Il master tira la linea "low" per almeno 480 µs.
3. Risposta del dispositivo (Presence Pulse):
   Dopo che il master rilascia la linea, il dispositivo DS18B20 attende circa 15–60 µs e poi tira la linea "low" per circa 60–240 µs per segnalare la propria presenza.

Trasmissione dei Dati: Write/Read Time Slots

La comunicazione avviene in "time slots" di circa 60–120 µs. Durante questi slot, il master:

- **Scrive:**
  - Per scrivere un bit "0": il master tira la linea "low" per la maggior parte dello slot (tipicamente ≥60 µs).
  - Per scrivere un bit "1": il master tira la linea "low" solo per un breve periodo (circa 1–15 µs) e poi la lascia alta.
- **Legge:**
  - Il master inizia il time slot abbassando la linea brevemente (circa 2 µs) e poi la rilassa. Dopo circa 15 µs, il master campiona il livello della linea. Se il dispositivo vuole inviare un "0", mantiene la linea bassa; per un "1", la linea è alta.



Comandi di Comunicazione

Il protocollo 1-Wire prevede una serie di comandi per gestire i dispositivi slave:

- **Comando “Match ROM” (0x55):** per selezionare un singolo dispositivo basandosi sul suo codice univoco.
- **Comando “Skip ROM” (0xCC):** per inviare comandi a tutti i dispositivi sul bus.
- **Comando “Convert T” (0x44):** per avviare la conversione della temperatura.
- **Comando “Read Scratchpad” (0xBE):** per leggere i dati memorizzati temporaneamente nel DS18B20.

1.2.2 Il Comando Convert T [44h] e lo Scratchpad

Funzionamento del **comando**:

Il comando Convert T (codice 0x44) è utilizzato per avviare una misura della temperatura:

- **Esecuzione:**
  Dopo aver inviato il comando 0x44 (tipicamente dopo aver selezionato il dispositivo con “Skip ROM” o “Match ROM”), il DS18B20 inizia la conversione analogico-digitale della temperatura.
- **Tempo di Conversione:**
  Il tempo necessario varia in base alla risoluzione impostata:
  - **9 bit:** circa 93.75 ms
  - **10 bit:** circa 187.5 ms
  - **11 bit:** circa 375 ms
  - **12 bit (default):** fino a 750 ms
    È possibile utilizzare la modalità “parasite power” per alimentare il dispositivo durante la conversione, oppure fornire una tensione esterna per ridurre le problematiche di timing.

Struttura dello **scratchpad**:
Il DS18B20 dispone di una memoria interna detta **Scratchpad** di 9 byte, in cui vengono memorizzati i dati di conversione e altre informazioni di configurazione.
La struttura tipica dello scratchpad è la seguente:

Byte	Descrizione
0	LSB della temperatura
1	MSB della temperatura
2	Registro di allarme (TL)
3	Registro di allarme (TH)
4	Registro di configurazione (risoluzione, ecc.)
5	Byte riservato (generalmente 0)
6	Byte riservato (generalmente 0)
7	Byte riservato
8	CRC (controllo di integrità dei dati)

Tabella che illustra la struttura dello Scratchpad del sensore. [2]

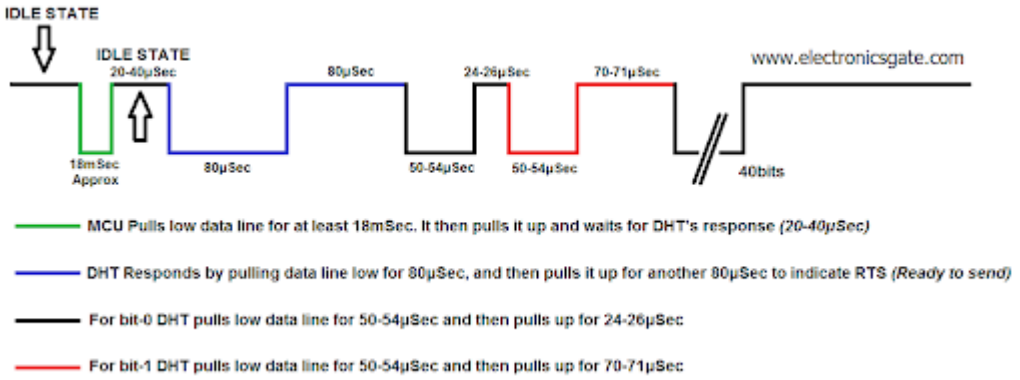
Nota: Dopo la conversione, il master deve inviare il comando “Read Scratchpad” (0xBE) per leggere questi 9 byte e ricavare il valore di temperatura, il quale viene codificato in formato a complemento a due.

Emulare il Protocollo 1-Wire sul DHT22

Il DHT22 è un sensore di temperatura e umidità che utilizza un protocollo a singolo filo, ma **con un funzionamento e tempi diversi** rispetto al DS18B20.
Per "emulare" il comportamento del DS18B20 (ovvero, avere una logica con "Scratchpad" e comando Convert 0x44) su un DHT22, si deve considerare quanto segue:

Differenze nei Protocolli

- **DS18B20 (1-Wire) [3]:**
  - Utilizza un bus aperto con tempi ben definiti per reset, scrittura e lettura.
  - Il comando Convert T (0x44) avvia una conversione interna che poi viene letta dallo scratchpad a 9 byte.
- **DHT22:**
  - Ha un proprio protocollo in cui il microcontrollore invia un segnale di inizio (solitamente abbassando il pin per almeno 1 ms) e poi il sensore risponde con una sequenza temporizzata di impulsi che rappresentano 40 bit di dati (16 bit per la temperatura, 16 bit per l'umidità, 8 bit di checksum).
  - I timing di invio e ricezione sono critici e differenti (tipicamente, la larghezza degli impulsi rappresenta "0" o "1").



Possibile Emulazione

Se si volesse "emulare" il DS18B20 sul DHT22, si dovrebbe implementare una logica che:

- **Intercetta il comando 0x44:**
  - Quando il master invia questo comando sul bus (configurato per seguire lo standard 1-Wire), il sistema deve riconoscerlo.
- **Avvia una "conversione" simulata:**
  - Invece di attivare una conversione interna, il sistema dovrebbe leggere i dati dal DHT22, tramite il suo protocollo nativo.
- **Popola uno "Scratchpad" virtuale:**
  - Creare una struttura dati in cui i dati rilevati dal DHT22 (temperatura e, eventualmente, umidità) sono mappati.

Schema dell'Emulazione

1. **Ricezione del Comando 0x44:**
    - Il firmware intercetta il comando Convert.
2. **Lettura del DHT22:**
    - Viene avviata una lettura del DHT22 usando il suo protocollo nativo.
    - Dopo aver ottenuto i dati (temperatura e umidità), il firmware li elabora.
3. **Popolamento dello Scratchpad Virtuale:**
    - I dati vengono mappati in una struttura dati formattata come lo scratchpad DS18B20.
4. **Lettura dello Scratchpad:**
    - Quando il master invia il comando "Read Scratchpad" (0xBE), il firmware restituisce la struttura virtuale, simulando un DS18B20. (nel caso specifico del progetto, visto che non c'è un vero e proprio *master*, la parte di lettura avverrà tramite stampa su monitor seriale su *Wokwi*, leggendo i dati dello scratchpad)

Considerazioni Pratiche

- **Precisione dei Timing:**
  Poiché entrambi i protocolli si basano su precise tempistiche, il firmware deve gestire accuratamente i timing per garantire la compatibilità con il master 1-Wire.
- **Utilizzo della Linea di Comunicazione:**
  Assicurarsi che il sistema che "emula" il DS18B20 gestisca la stessa fisica (ad esempio, la modalità open-drain e il pull-up esterno) per non creare conflitti con il master.

Conclusioni

- **DS18B20 e 1-Wire:**
  Il DS18B20 utilizza un protocollo 1-Wire che prevede reset, presenza, comunicazione in time slot e una serie di comandi (tra cui il Convert T 0x44) che attivano la conversione della temperatura e il successivo accesso a uno scratchpad di 9 byte.
- **Emulazione su DHT22:**
  Sebbene il DHT22 utilizzi un protocollo completamente diverso, si può "emulare" il comportamento del DS18B20 creando un livello di astrazione software/hardware che intercetti i comandi 1-Wire, converta il comando Convert 0x44 in una lettura nativa del DHT22, e restituisca i dati in un formato simile allo scratchpad del DS18B20.

1.2.3 Obiettivi e Contesto

Arrivati a questo punto si può evincere che l'obiettivo del progetto è realizzare un sistema che, partendo dalla ricezione del comando 1-Wire "Convert T" [44h], attivi una conversione "virtuale" che consiste nella lettura dei dati da un sensore DHT22, e che successivamente consenta al master di leggere i dati formattati in uno "scratchpad" (simile a quello del DS18B20).

1.2.3.1 Elementi Computazionali e Specifiche Numeriche

1. **Tempi di Conversione e Timing:**
    - **Comando Convert (0x44):**
      Il comando attiva la conversione.

      Nel firmware che verrà sviluppato per soddisfare la consegna, la conversione (simulata) durerà almeno 50 ms.
      Il codice dovrà emulare il comportamento del sensore DS18B20 (usato nello schema 1-Wire), dove dopo il comando Convert T (0x44), il sensore impiega del tempo per completare la misurazione. Nel DS18B20, questo tempo varia da 94ms a 750ms a seconda della risoluzione.

      In questo caso specifico, però, si potrebbe scegliere un tempo di 50ms per semplificare la simulazione del DHT22, che in realtà ha un protocollo diverso ma viene qui adattato al contesto 1-Wire.

      Senza questa attesa, la funzione di lettura verrebbe eseguita immediatamente dopo il comando, rischiando di leggere dati non pronti o residui di una misurazione precedente.

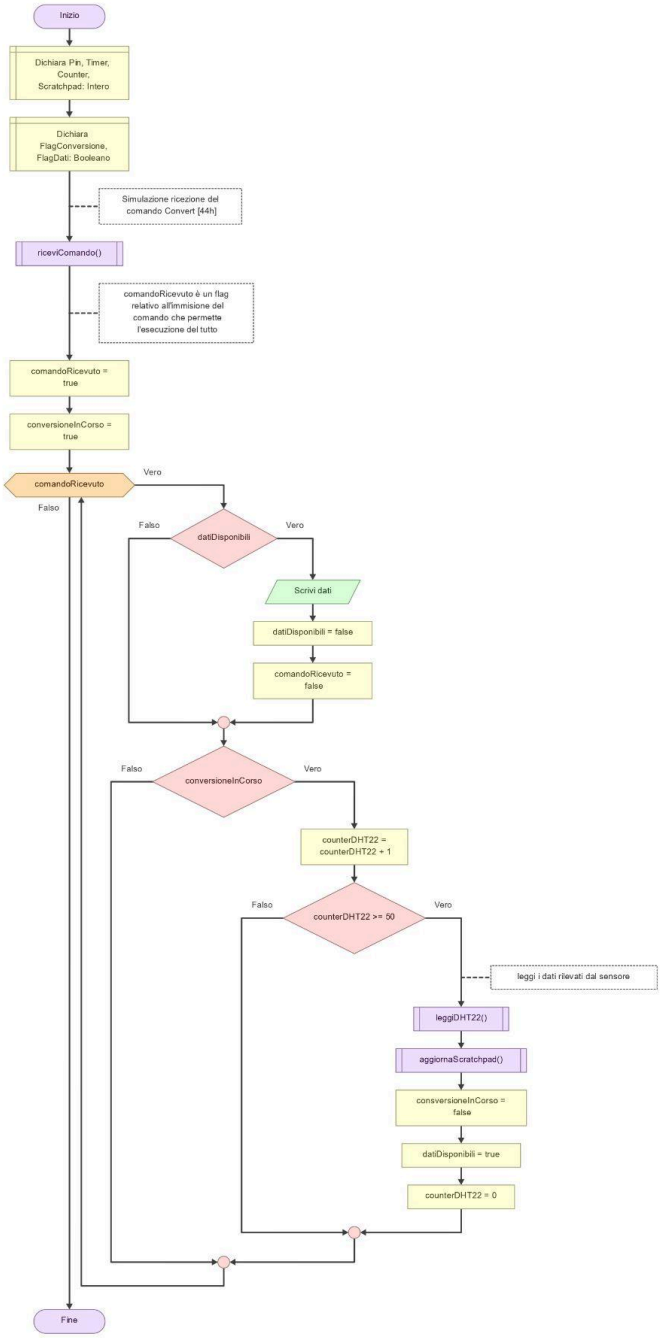
      Usare un timer con interrupt permette di attendere senza bloccare il loop principale. Il microcontroller può eseguire altre operazioni durante l'attesa.

2. **Input e Output:**
    - **Input:**
        - **Segnale di comando 1-Wire:** In questo caso, il firmware simulerà la ricezione del comando Convert (0x44) in modo "hardcoded" nel loop, ma in un'applicazione reale questo comando verrebbe ricevuto sul bus 1-Wire.
        - **Dati dal DHT22:** I dati fisici letti dal sensore DHT22 includono 40 bit distribuiti in 5 byte (umidità alta, umidità bassa, temperatura alta, temperatura bassa e checksum).
    - **Output:**
        - **Scratchpad Virtuale:** Una struttura dati in cui vengono salvati i valori di temperatura e umidità.
        - **Monitor Serial (Wokwi):** Visualizzazione dei dati letti e formattati (temperatura in °C e umidità in %).

3. **Casi Estremi e Vincoli:**
    - **Vincoli Fisici del DHT22:**
        - Il protocollo nativo del DHT22 richiede una sequenza di inizializzazione che consiste nel portare il pin basso per almeno 1 ms, seguita dall'attesa della risposta del sensore.
        - I tempi per la lettura dei bit sono critici: ad esempio, la larghezza degli impulsi è usata per distinguere un "0" da un "1".
    - **Vincoli del Protocollo 1-Wire:**
        - Gestione del reset, presenza e comunicazione a slot temporali.
        - In questo progetto il "comando" Convert viene interpretato dal firmware e mappato su una conversione con il DHT22.
    - **Errori e Checksum:**
        - Il firmware dovrà includere una verifica del checksum (data[4] = somma dei primi 4 byte & 0xFF) per assicurare che i dati letti siano validi.

1.2.4 Flusso Logico (Flowchart Concettuale)

Di seguito un possibile flowchart descrittivo del sistema:



Modularità e Isolamento dei Componenti Logici:

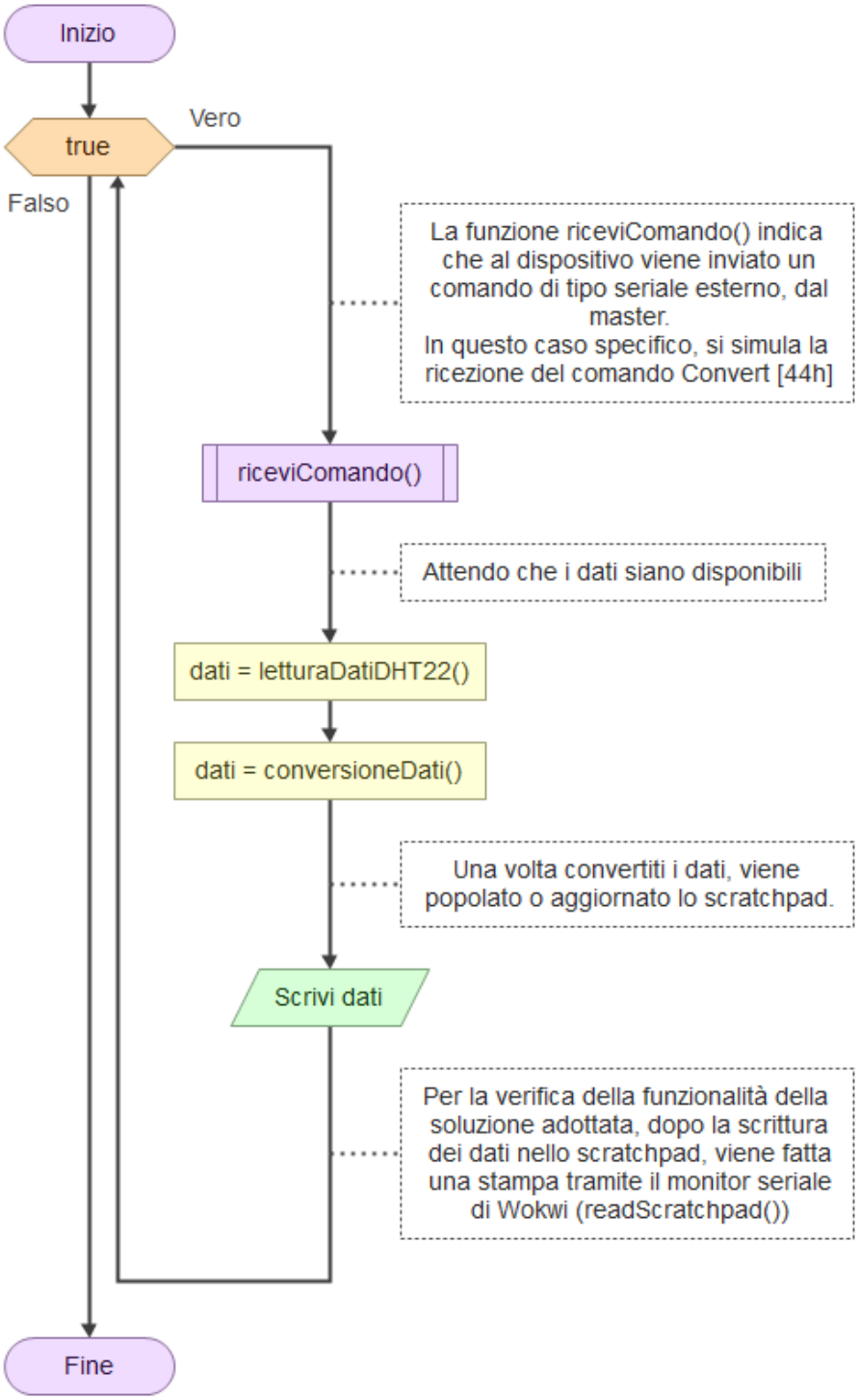
Il sistema può essere suddiviso in moduli indipendenti:

- **Modulo Timer:** Gestisce la temporizzazione e gli interrupt per attivare la lettura.
- **Modulo Comando 1-Wire:** Funzione `riceviComando()` che interpreta i comandi (in questo caso, solo il comando Convert T [44h]).
- **Modulo Lettura DHT22:** Funzione `leggiDHT22()` che si occupa di inizializzare la comunicazione con il sensore, leggere 40 bit e aggiornare lo scratchpad.
- **Modulo Visualizzazione:** Gestito nel loop principale tramite la stampa su monitor seriale.

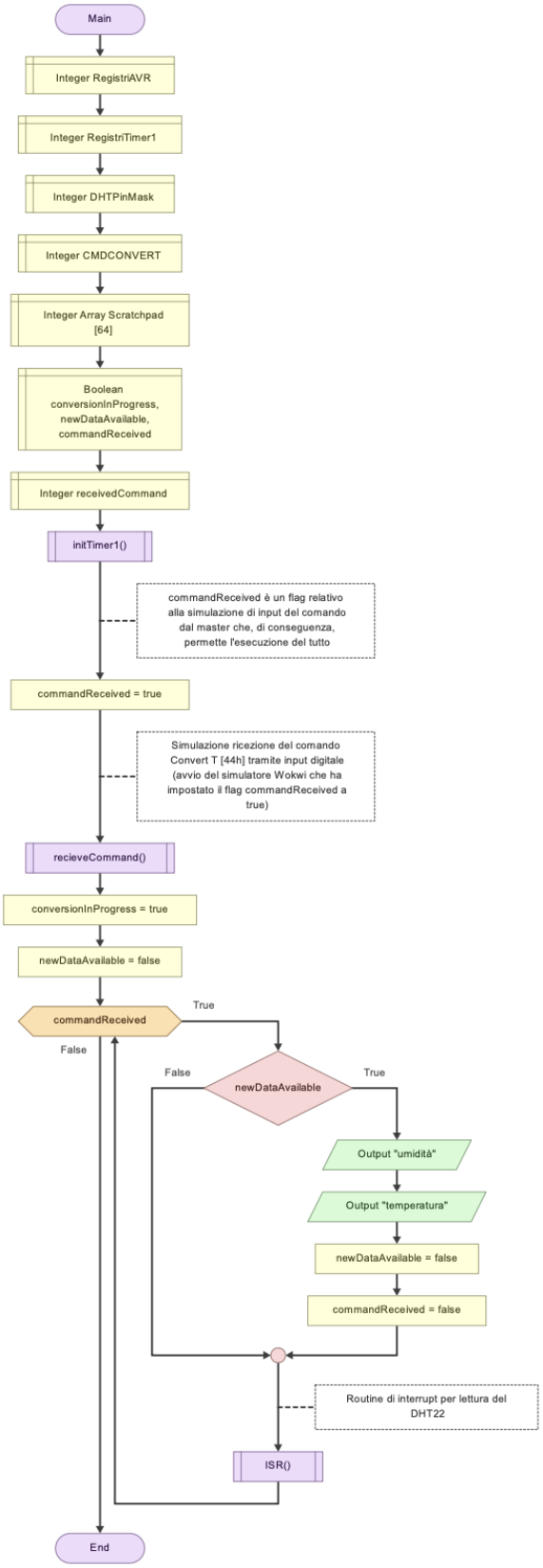
Questi moduli operano su variabili di input (timing, stato del pin, dati del DHT22) e restituiscono variabili di output (valori di temperatura e umidità, flag che indicano la presenza di nuovi dati).

1.3 Analisi della soluzione adottata

1.3.1 Flowchart generale



1.3.2 Flowchart tecnico descrittivo del firmware



Con questo flowchart è stato rappresentato quale dovrà essere il funzionamento complessivo del firmware eseguito dalla scheda microcontrollore.

Ci sono diversi **elementi** cruciali che descrivono il flusso degli eventi:

- Dichiarazione registri AVR
- Dichiarazione dei registri del Timer
- Dichiarazione del pin a cui è collegato il sensore
- Definizione comando Convert
- Definizione struttura scratchpad
- Dichiarazione e inizializzazione dei flag che gestiscono il cambio di stato
- L'inizializzazione del Timer1
- La ricezione del comando da parte del master e interpretazione di esso
- Routine di Interrupt
    - Funzione per la lettura dei dati rilevati dal sensore e successivo aggiornamento dello scratchpad
- Output dati, reset dei flag

- **Definizione registri AVR e del Timer**
  - Si definiscono gli indirizzi dei registri di I/O del microcontrollore ATMega328P (presente sull'Arduino Uno). Questi registri controllano le porte digitali e il Timer1. Dichiarati con *volatile* perché questi registri possono essere modificati al di fuori del flusso principale del programma (es. da un interrupt):
    - **DDRD**: *Data Direction Register per la porta D*.
      - Serve a configurare i pin della porta D come input o output. Scrivendo un 1 su un bit, il pin corrispondente diventa un output. Scrivendo uno 0 il pin diventa un input.
    - **PORTD**: *Data Register per la porta D*.
      - Se un pin è configurato come output, scrivere un 1 su un bit imposta il pin corrispondente a HIGH, mentre scrivere uno 0 lo imposta a LOW. Se un pin è configurato come input e il pull-up interno è abilitato, scrivere un 1 abilita il pull-up.
    - **PIND**: *Input Pins Address per la porta D*.
      - Leggendo un bit di questo registro, si ottiene lo stato logico attuale del pin corrispondente (HIGH o LOW).
    - **TCCR1B**: *Timer/Counter1 Control Register B*.
      - Controlla il funzionamento del Timer1, inclusa la modalità di funzionamento e il prescaler.
    - **TIMSK1**: *Timer/Counter1 Interrupt Mask Register.*
      - Abilita o disabilita gli interrupt generati dal Timer1.
    - **OCR1A**: *Timer/Counter1 Output Compare Register A*.
      - Utilizzato in modalità CTC (Clear Timer on Compare Match) per impostare il valore di confronto. Quando il contatore del Timer1 raggiunge il valore in OCR1A, viene generato un interrupt e il contatore viene resettato.

- **Dichiarazione del pin a cui è collegato il sensore**
  - Viene definita una maschera di bit per il pin 2 della porta D (PD2). Questo è il pin che verrà utilizzato per "comunicare" con il sensore DHT22 simulato. La maschera 0x04 corrisponde al bit 2 (0b00000100).

- **Definizione comando Convert**
  - Definizione del codice del comando Convert T (0x44), che simula il comando che un master 1-Wire invierebbe a un DS18B20 per avviare una conversione di temperatura.

- **Definizione struttura scratchpad**
    - Questa struttura definisce il formato dei dati che saranno "letti" dal master 1-Wire.
        - Contiene la temperatura e l'umidità, entrambe memorizzate come interi a 32 bit e moltiplicate per 10 (per avere una cifra decimale).

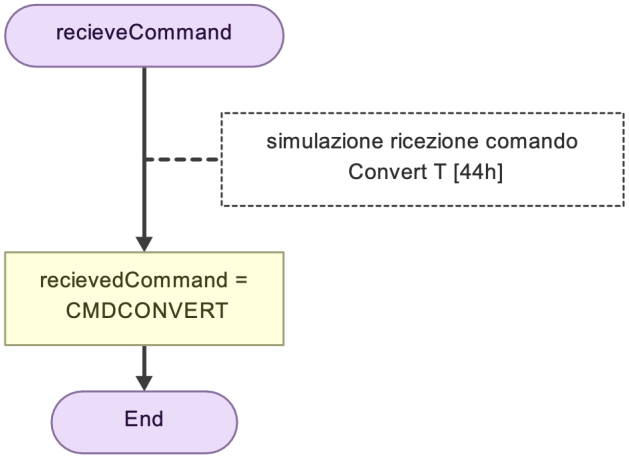
- **Dichiarazione e inizializzazione dei flag che gestiscono il cambio di stato**
    - `conversionInProgress` : Un flag che indica se la conversione (simulata) è in corso.
    - `newDataAvailable` : Un flag che indica se ci sono nuovi dati disponibili nello scratchpad.
    - `commandReceived` : Un flag che indica se è stato ricevuto un comando.
        - In questo codice, è inizializzato a true per avviare immediatamente la prima conversione, per far si che la simulazione sia fedele alla richiesta.
        - Nel codice finale, questo flag dovrebbe essere gestito dalla logica di ricezione del comando 1-Wire.

- **Output dati, reset dei flag**
    - Nella sezione loop, una volta che il flag `newDataAvailable` è impostato a true, si verifica la veridicità dei dati contenuti nello scratchpad facendo una stampa su monitor seriale
    - Il flag `newDataAvailable` viene impostato a false.
    - Il flag `commandReceived` viene impostato a false.

Nelle pagine successive vengono illustrate nel dettaglio, tramite altri flowchart, le seguenti funzioni:

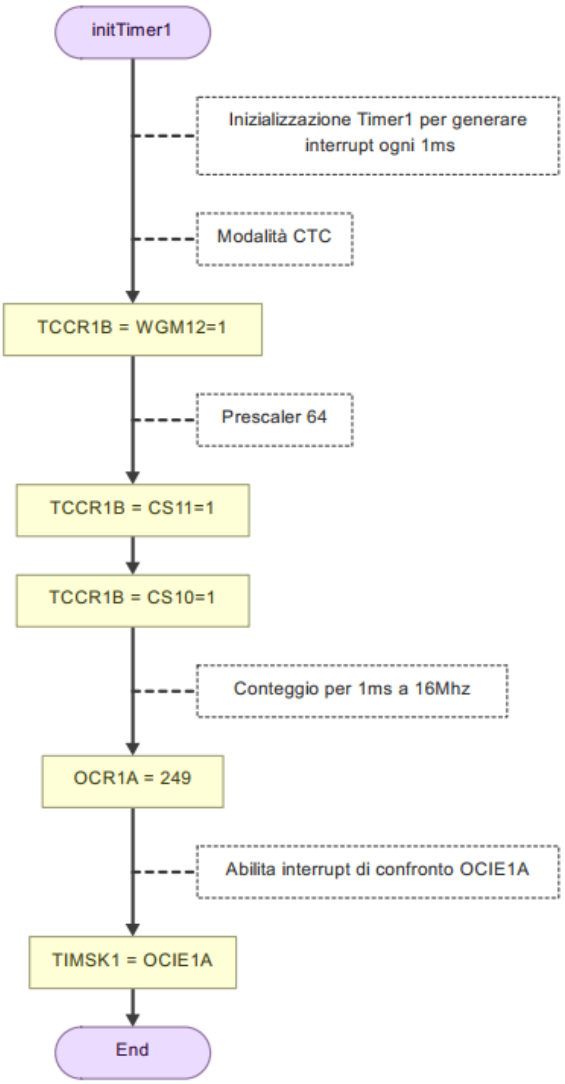
- Funzione per ricezione comando inviato dal master e interpretazione
- Funzione inizializzazione Timer
- Routine di interrupt
- Funzione per la lettura dei dati rilevati dal sensore e successivo aggiornamento dello scratchpad (readDHT22)

1.3.3 Flowchart ricezione comando inviato dal master e interpretazione



È stata inserita una funzione che gestisce i comandi ricevuti dal master 1-Wire. In questo caso, l'unico comando supportato è `CMD_CONVERT (0x44)`.
Quando viene ricevuto questo comando, la funzione chiama `startConversion()` per avviare la conversione simulata.
La funzione `startConversion()` imposta il flag `conversionInProgress` a true se la conversione non è già in corso. Questo flag indica all'ISR di avviare il processo di "lettura" del DHT22 simulato.

1.3.4 Flowchart funzione inizializzazione Timer



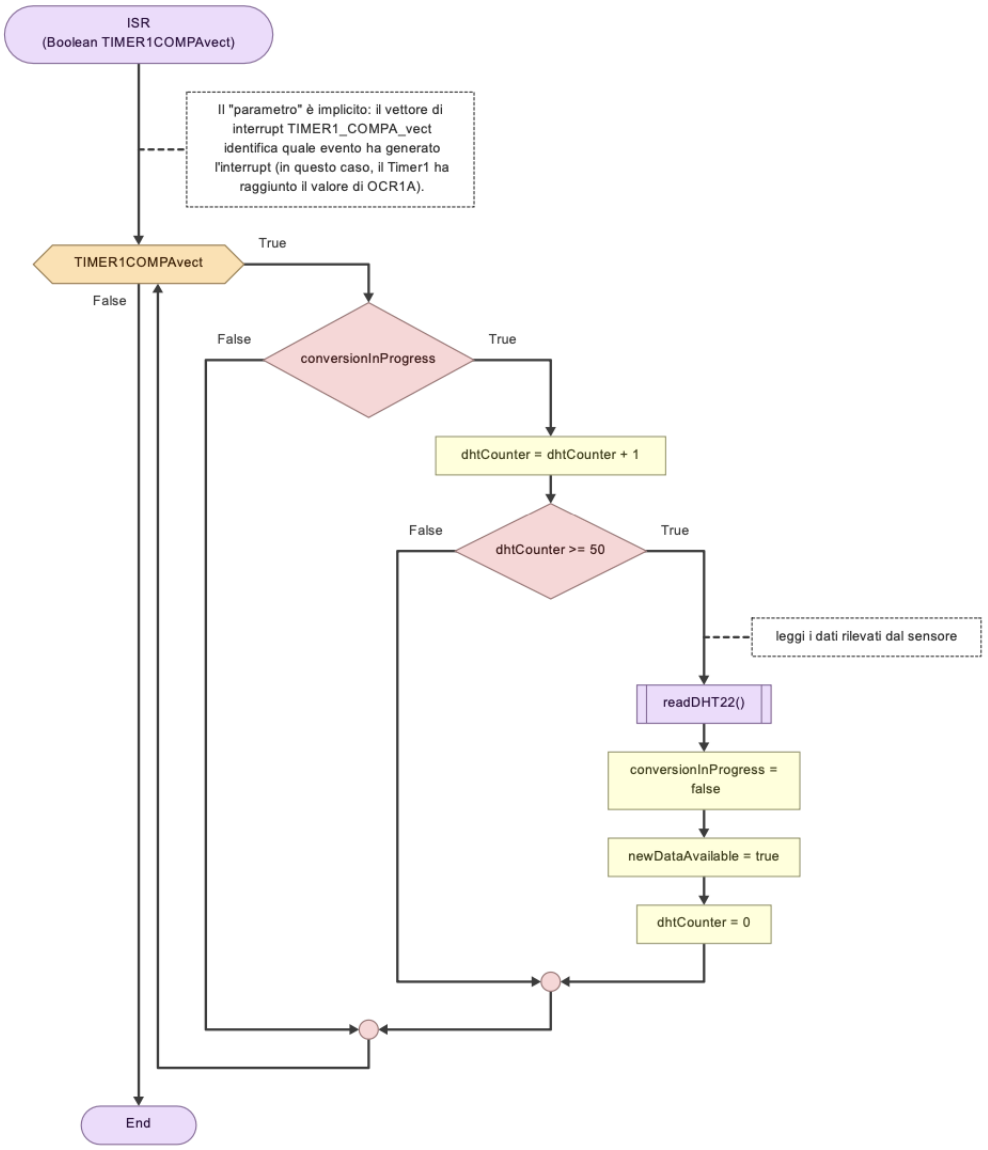
Questa funzione inizializza il Timer1 per generare un interrupt ogni 1 millisecondo. Questo interrupt viene utilizzato per temporizzare la "lettura" del DHT22 simulato.

```
TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10);
```
Imposta il Timer1 in modalità CTC (Clear Timer on Compare Match) con un prescaler di 64.
La modalità CTC fa sì che il timer si resetti quando il suo contatore raggiunge il valore memorizzato in OCR1A.
Il prescaler divide la frequenza di clock del microcontrollore (16 MHz) per 64.

```
OCR1A = 249;
```
Imposta il valore di confronto a 249.
Con un prescaler di 64 e una frequenza di clock di 16 MHz, il timer incrementa a una frequenza di 16 MHz / 64 = 250 kHz. Impostando OCR1A a 249, l'interrupt viene generato ogni 250 cicli di clock, che corrispondono a 1 ms (250 / 250 kHz = 0.001 s).

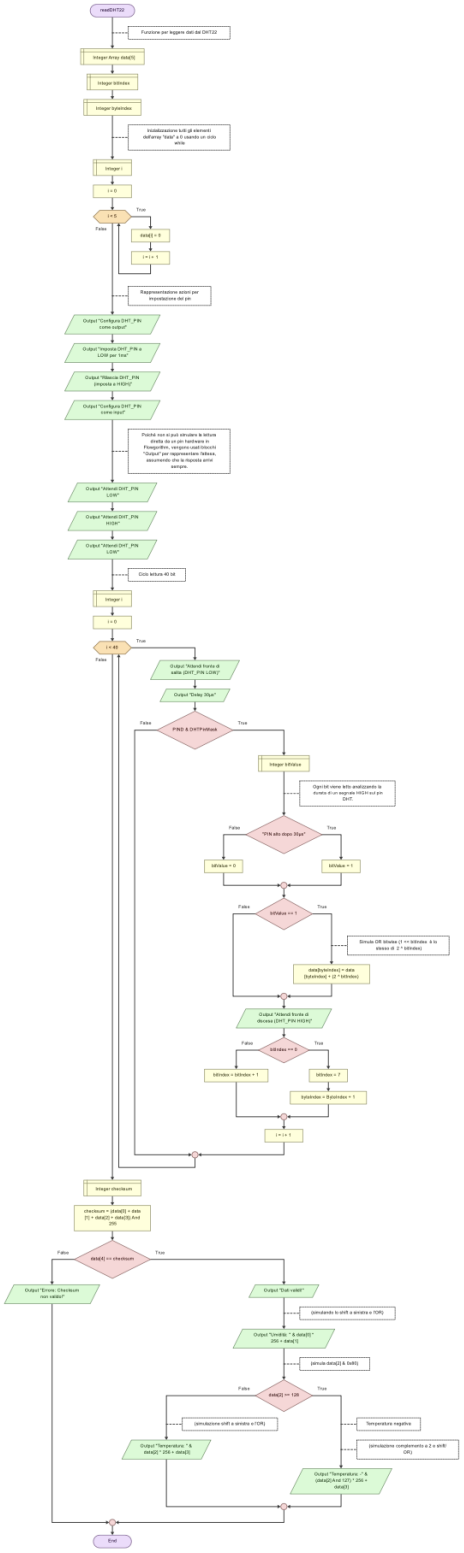
```
TIMSK1 = (1 << OCIE1A);
```
Abilita l'interrupt di confronto A (Output Compare A Interrupt Enable) per il Timer1.

Flowchart routine di interrupt



La routine di interrupt viene eseguita ogni volta che il Timer1 raggiunge il valore di OCR1A (ogni 1 ms):
Viene istanziata una variabile statica per contare il tempo trascorso dall'inizio della conversione simulata.
Si controlla se la conversione è in corso verificando che `conversionInProgress` sia true.
Il contatore viene incrementato ogni millisecondo.
Verifica se sono trascorsi 50 millisecondi (il tempo stimato per la conversione del DHT22).
Viene chiamata la funzione `readDHT22()` per "leggere" i dati del DHT22 simulato.
Si imposta il flag `newDataAvailable` a true per indicare che ci sono nuovi dati disponibili.
Il flag `conversionInProgress` viene impostato a false per indicare che la conversione è terminata.
Il contatore viene resettato dopo queste operazioni.

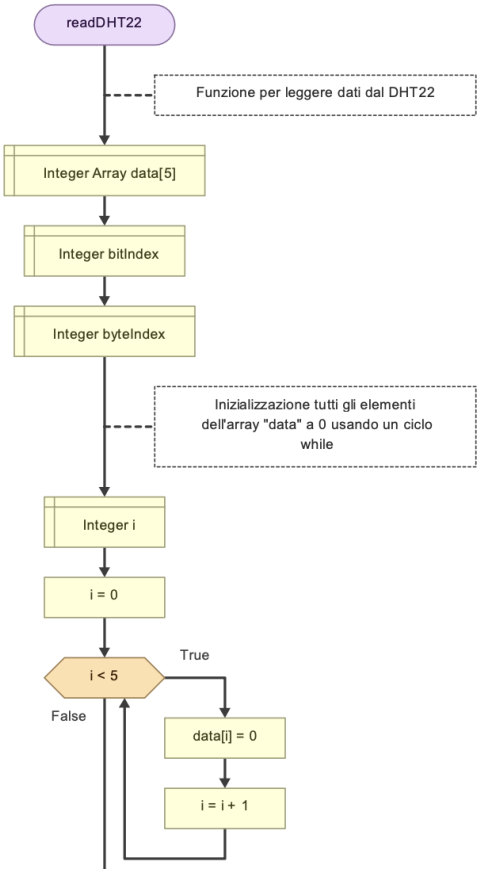
1.3.5 Flowchart funzione per la lettura dei dati rilevati dal sensore e successivo aggiornamento dello scratchpad (readDHT22)



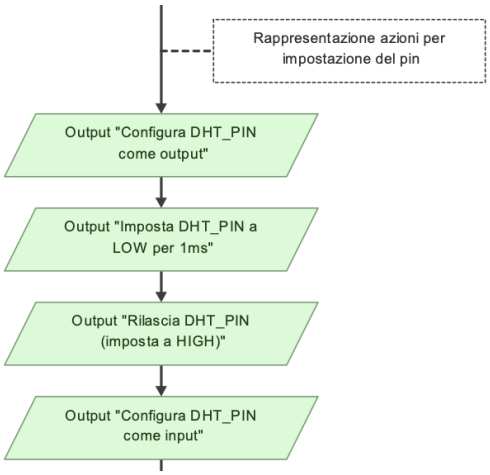
Questa funzione simula la lettura dei dati dal sensore DHT22.

Genera una sequenza di impulsi sul pin definito da **DHT_PIN_MASK** per simulare la comunicazione e poi interpreta questa sequenza per ricavare i dati.

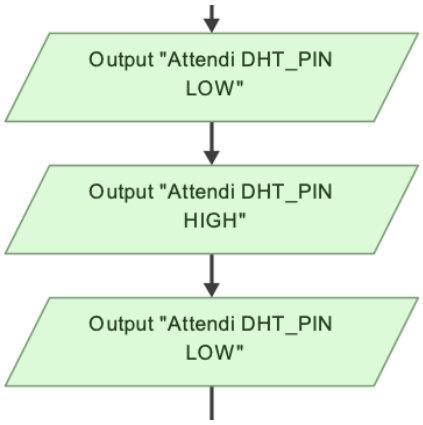
Nelle pagine successive viene illustrata passo per passo.



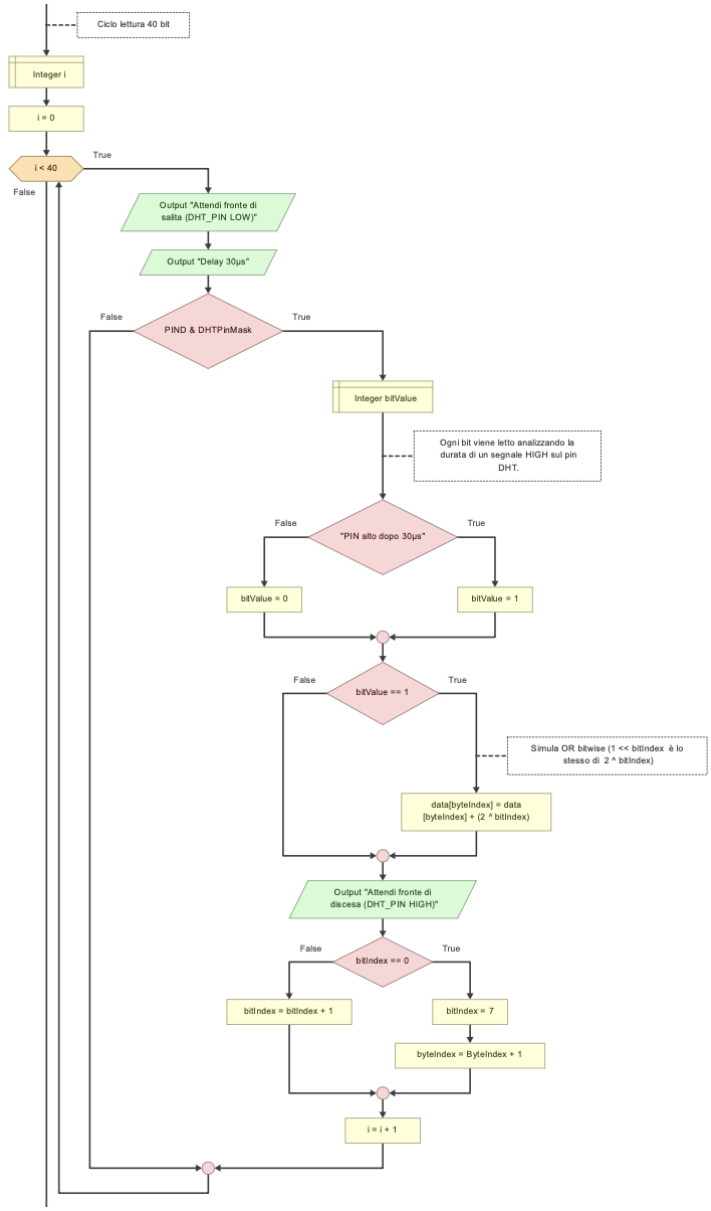
Dopo che viene invocato il metodo `readDHT22()`, Si dichiara un **array di 5 byte** per memorizzare i dati ricevuti dal sensore e due variabili per gestire l'indice del bit e del byte durante la lettura. Successivamente, l'array viene inizializzato.



Inizializzazione comunicazione con il sensore (Simulazione):
Questa sezione simula la sequenza di inizializzazione della comunicazione con il DHT22.
Si imposta il pin come output, lo porta a LOW per 1ms, lo rilascia e lo riconfigura come input.



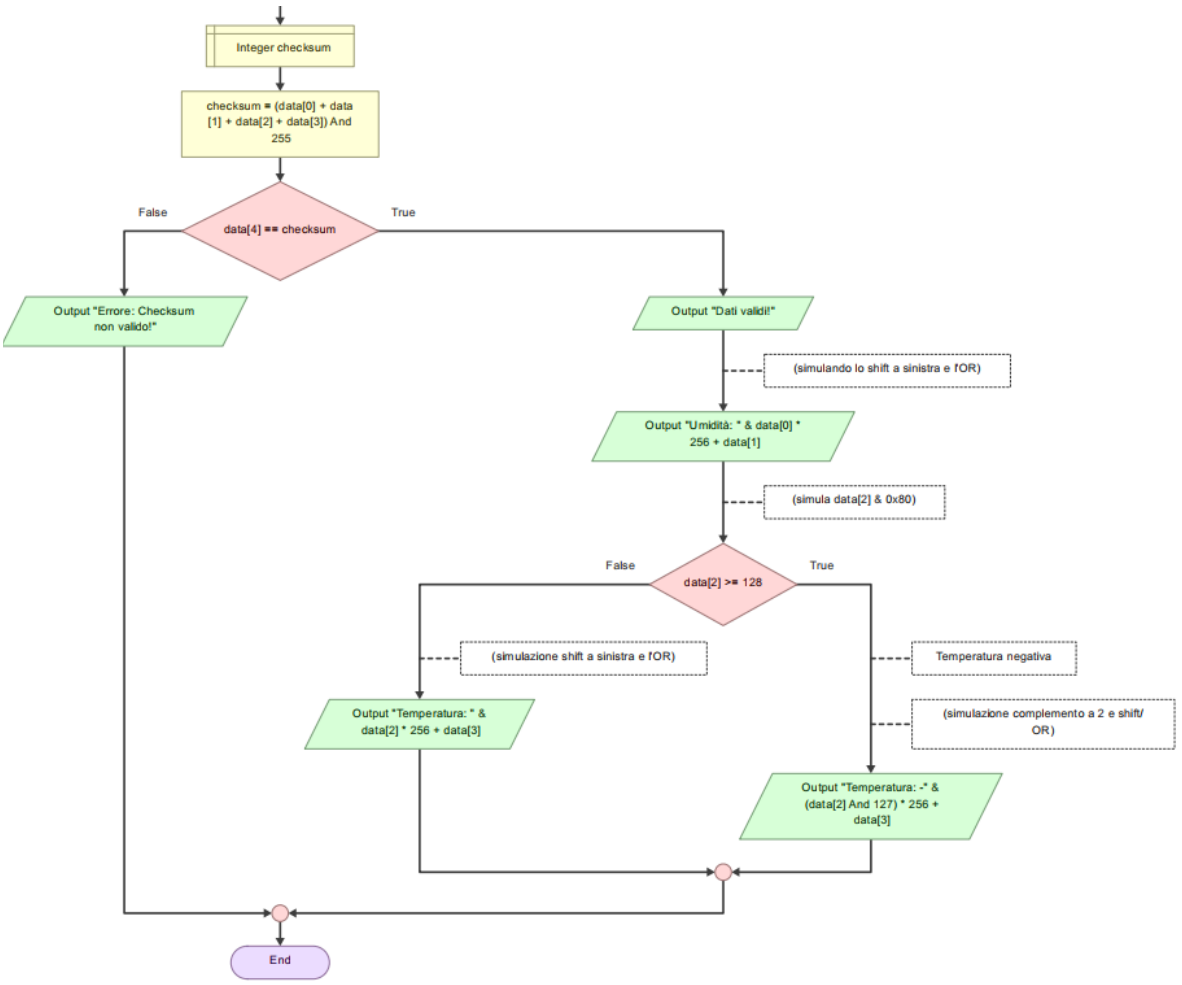
Attesa risposta del sensore (Simulazione):
L'Arduino attende i segnali di risposta dal sensore. In questo caso, si attende una sequenza di livelli HIGH e LOW sul pin.



Lettura di 40 bit di dati:

Questa sezione rappresenta la lettura dei 40 bit di dati dal DHT22.

Il ciclo for itera 40 volte, e per ogni iterazione, il codice attende un fronte di salita, attende 30 microsecondi, legge il valore del pin (che determina se il bit è 0 o 1), e attende un fronte di discesa.



Verifica checksum e aggiornamento scratchpad (Simulazione):
Questa sezione verifica il checksum per validare i dati ricevuti.
Il checksum viene calcolato sommando i primi quattro byte dei dati e confrontandolo con il quinto byte.
Se il checksum è corretto, i dati vengono considerati validi.

Aggiornamento scratchpad:
Se il checksum è valido, i dati di umidità e temperatura vengono estratti dall'array data e memorizzati nello scratchpad.
La temperatura viene gestita correttamente anche nel caso sia negativa.

Scelte Progettuali e Risorse Impiegate

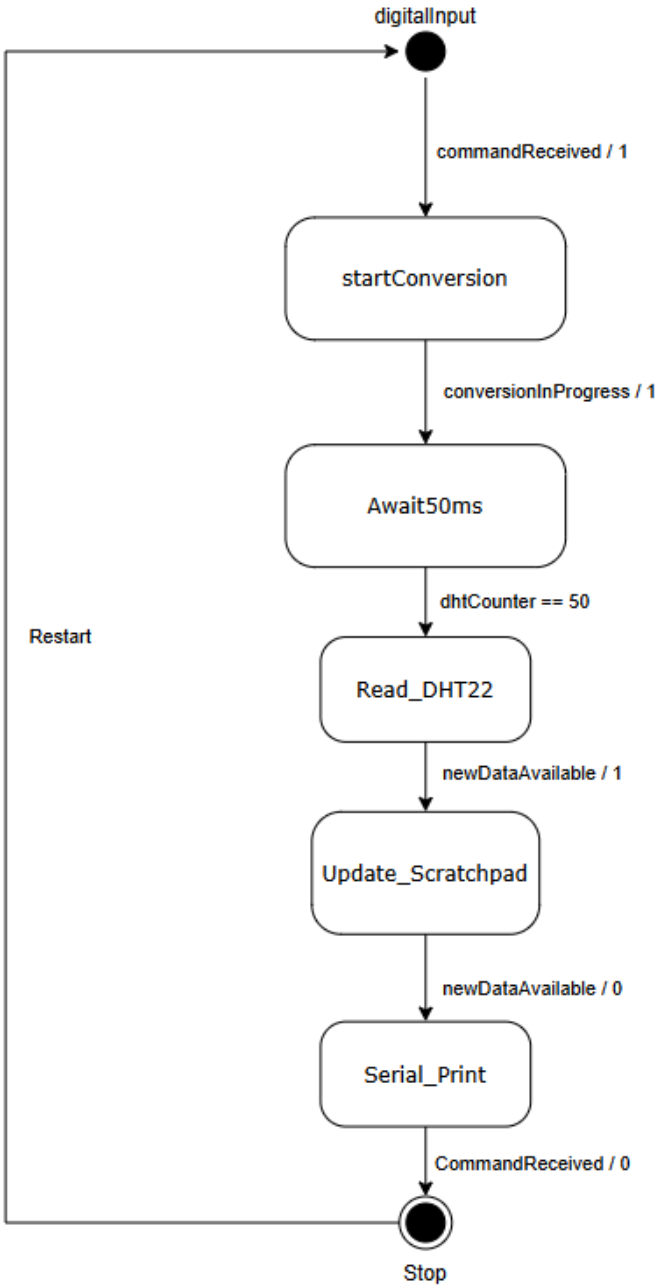
La soluzione adottata è stata progettata per:

- **Minimizzare l'utilizzo delle risorse:**
  - L'uso del Timer1 in modalità CTC con prescaler riduce il carico sul microcontrollore.
  - La gestione tramite interrupt consente di separare il timing dalla logica del loop principale.
- **Massimizzare le prestazioni e la precisione temporale:**
  - L'uso di un ISR per il controllo del tempo (1 ms) garantisce il rispetto dei tempi critici del protocollo DHT22.
  - La simulazione del comando Convert (0x44) permette di mappare il comportamento del DS18B20, mantenendo una struttura compatibile con il protocollo 1-Wire.

1.3.6 Statechart complessivo

Legenda:

Sulle transitions ho utilizzato: variabile che consente il cambio di stato / valore assunto



Moduli di Elaborazione

1. **digitalInput:**
   - **Ingresso:** Input digitale da parte del compilatore del simulatore (Wokwi).
   - **Elaborazione**: Flag `commandReceived` impostato a true durante l'inizializzazione.
   - **Uscita:** Attivazione della variabile `commandReceived`.
2. **startConversion:**
   - **Ingresso:** Flag `commandReceived` impostato a true.
   - **Elaborazione:** aggiornamento valore del flag `conversionInProgress` che indica l'inizio della conversione.
   - **Uscita:** Flag `commandReceived` impostato a true.
3. **Await50ms:**
   - **Ingresso:** Flag `commandReceived` impostato a true.
   - **Elaborazione:** Incremento del contatore dhtCounter all'interno della routine dell' Interrupt Service Routine;
   - **Uscita:** Al raggiungimento del tempo limite (50 ms), viene chiamata readDHT22().
4. **Read_DHT22:**
   - **Ingresso:** Chiamata da parte dell'ISR.
   - **Elaborazione:** Campionamento dei bit con controlli di timing e verifica checksum.
   - **Uscita:** Dati rilevati e convalidati, flag `conversionInProgress` impostato a false.
5. **Update_Scratchpad:**
   - **Ingresso:** Dati rilevati dalla lettura del sensore.
   - **Elaborazione:** Aggiornamento dello scratchpad.
   - **Uscita:** Scratchpad aggiornato, flag `newDataAvailable` impostato a true.
6. **Serial_Print:**
   - **Ingresso:** Variabile `newDataAvailable` impostata a true e dati presenti nello scratchpad.
   - **Elaborazione:** Conversione dei valori (dividendo per 10 per ottenere i valori reali).
   - **Uscita:** Stampa su monitor seriale (in Wokwi), flag `newDataAvailable` impostato a false.
7. **Stop:**
   - **Ingresso:** Flag `commandReceived` impostato a false.
   - **Elaborazione:** Controllo delle condizioni per continuare, eventualmente, l'esecuzione del comando.
   - **Uscita:** Il sistema ora è in attesa di un nuovo comando.
8. **Restart:**
   - Ho inserito questo modulo per specificare che bisogna riavviare il simulatore per simulare nuovamente l'input digitale.

Dipendenze Funzionali e Connessioni

- **Eventi:**
  - L'arrivo del comando Convert (simulato) innesca la conversione.
  - L'interrupt del Timer1 sincronizza la lettura.
- **Variabili:**
  - `conversionInProgress` determina l'avvio e la fine della conversione.
  - `commandReceived` è un flag fittizio inserito per simulare l'avvenuta ricezione del comando e che, poi, permette di arrestare l'esecuzione del programma, per evitare una "ricezione" continua del comando Convert t [44h] dovuta alla sua presenza nel loop.
  - `newDataAvailable` segnala quando la lettura dei valori di temperatura e umidità è avvenuta e lo scratchpad è stato aggiornato.
  - `scratchpad` memorizza i dati formattati, fungendo da interfaccia tra la lettura del DHT22 e la visualizzazione.
- **Connessioni:**
  - La funzione `processCommand(uint8_t cmd)` è connessa al modulo Timer, tramite la funzione `startConversion()` (che simula la ricezione e l'esecuzione del comando).
  - Il modulo Timer invoca `readDHT22()` tramite il controllo del contatore, integrando la logica di timing.

1.4 Analisi delle soluzioni alternative

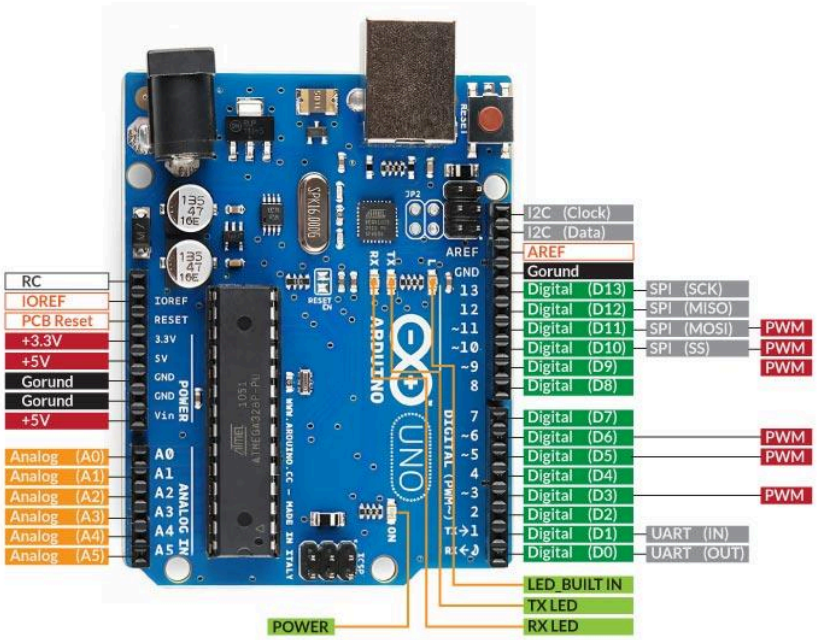
1. **Uso di librerie Arduino:**
   - Scartato per non rispettare i requisiti del progetto (gestione diretta degli I/O).
2. **Uso di funzioni standard per il sensore:**
   - Scartato per l'obbligo di implementare un sistema personalizzato.

2. Sintesi

2.1 Sintesi componente ME1

Arduino Uno:

- Arduino Uno è una scheda microcontrollore open source estremamente versatile e facile da usare, ideale per prototipazione e progetti embedded. Ecco alcuni aspetti chiave:
  - **Microcontrollore**: Basato su ATmega328P, che opera a 16 MHz, garantisce prestazioni adeguate per la maggior parte delle applicazioni.
  - **I/O**: Dispone di 14 pin digitali (6 dei quali supportano il PWM) e 6 ingressi analogici, offrendo ampia flessibilità per interfacciarsi con sensori, attuatori e altri moduli.
  - **Connettività**: Il collegamento USB permette sia la programmazione che l'alimentazione, semplificando il processo di sviluppo.
  - **Alimentazione**: Funziona a 5V, rendendolo compatibile con numerosi sensori e componenti.
- In questo caso specifico funge da master e microcontrollore principale:
  - Esegue il firmware che emula il protocollo 1-Wire, gestisce la temporizzazione (tramite Timer1) e coordina la lettura del sensore.
  - Comunica con il sensore DHT22 tramite un pin digitale.
  - Visualizza i dati (temperatura e umidità) sul monitor seriale.



Pinout e specifiche dell'Arduino Uno [4]

2.2 Sintesi componente ME2

Sensore DHT22:

- Il sensore di temperatura e umidità **DHT22** utilizza al suo interno un sensore di umidità capacitivo e un termistore per misurare l'aria circostante, e comunica le sue misurazioni tramite un segnale digitale sul pin dei dati.
- Il componente di rilevamento dell'**umidità** è formato da due elettrodi con un substrato che trattiene l'umidità (un polimero di plastica conduttivo o di sale) tra loro. Gli ioni vengono rilasciati dal substrato quando il vapore acqueo viene assorbito da esso, il che a sua volta aumenta la conduttività tra gli elettrodi. La variazione di resistenza tra i due elettrodi è proporzionale all'umidità relativa. Una maggiore umidità relativa diminuisce la resistenza tra gli elettrodi, mentre una minore umidità relativa aumenta la resistenza tra gli elettrodi.
- La **temperatura** viene invece misurata tramite un termistore NTC, il termistore è un resistore termico, un resistore che cambia la sua resistenza con la temperatura. Tecnicamente, tutti i resistori sono termistori – la loro resistenza cambia leggermente con la temperatura – ma la variazione è solitamente molto piccola e difficile da misurare.
  I termistori invece sono realizzati in modo che la resistenza cambi drasticamente con la temperatura in modo che possa essere di 100 ohm o più di variazione per grado.
  Il termine "NTC" significa "coefficiente di temperatura negativo", il che significa che la resistenza diminuisce con l'aumentare della temperatura.

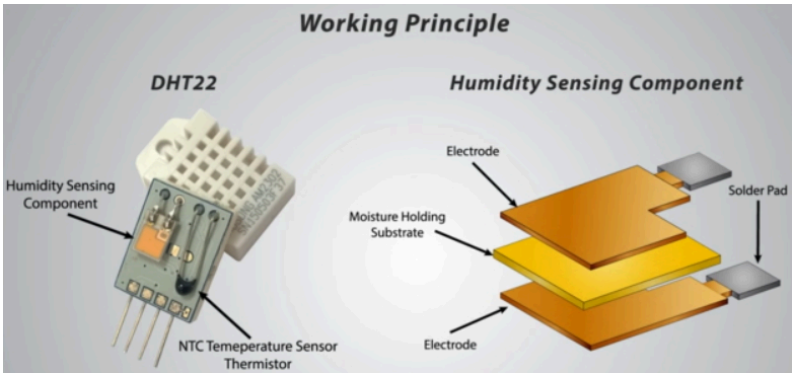
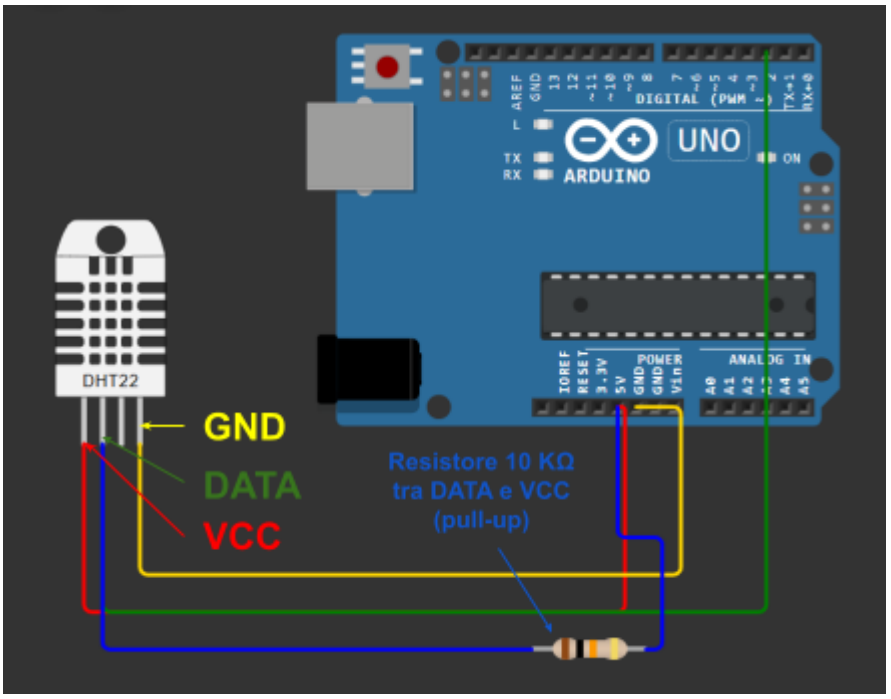


Immagine che mostra l'interno del DHT22 e i vari layer sensore dell'umidità [1]

Collegamento del sensore con Arduino Uno:

- Punti di Connessione:
  - **VCC:** Alimentato a +5V.
  - **DATA:** Collegato a un pin digitale dell'Arduino per la trasmissione dei dati.
  - **GND:** Connesso alla massa.
  - **Resistore di pull-up:** Collegato con linea dati e alimentazione
- Alimentazione:
  - Il sensore DHT22 viene alimentato a +5V, che viene fornito dal microcontrollore e collegato al pin VCC del DHT22.
- Resistore di Pull-Up:
  - La linea dati del DHT22 è collegata ad un resistore di pull-up tra +5V e la linea dati stessa. Questo è fondamentale per garantire che il livello logico in assenza di trasmissione sia alto e per stabilizzare il segnale, in quanto il DHT22 utilizza una comunicazione a singolo filo.
  - In questo caso specifico è stata utilizzato un resistore da 10 KΩ
- Linea Dati:
  - La linea dati (DATA) del DHT22 è connessa a un pin digitale dell'Arduino (nel firmware viene usato il PD2, ovvero il pin 2), che viene usato per inviare il segnale di inizializzazione e per leggere i dati dal sensore.
- Messa a Terra (GND):
  - Il GND del DHT22 è collegato alla massa (GND) dell'Arduino, completando il circuito.



Implementazione effettiva del circuito di collegamento fra il sensore l'arduino [rif.]

3. Sistema complessivo

Il progetto è fruibile su Wokwi, tramite il seguente link:
https://wokwi.com/projects/420233210795848705

3.1 Manuale utilizzo

Sopracitato, è presente l'uri del progetto implementato su Wokwi e, una volta arrivati alla pagina specifica della piattaforma, basterà semplicemente cliccare sul pulsante di "**Avvio**" della simulazione (che imita l'invio del comando Convert T [44h]da master a slave, quindi un input digitale) e si potranno visualizzare a schermo, tramite il monitor seriale della piattaforma, i valori rilevati dal sensore DHT22.

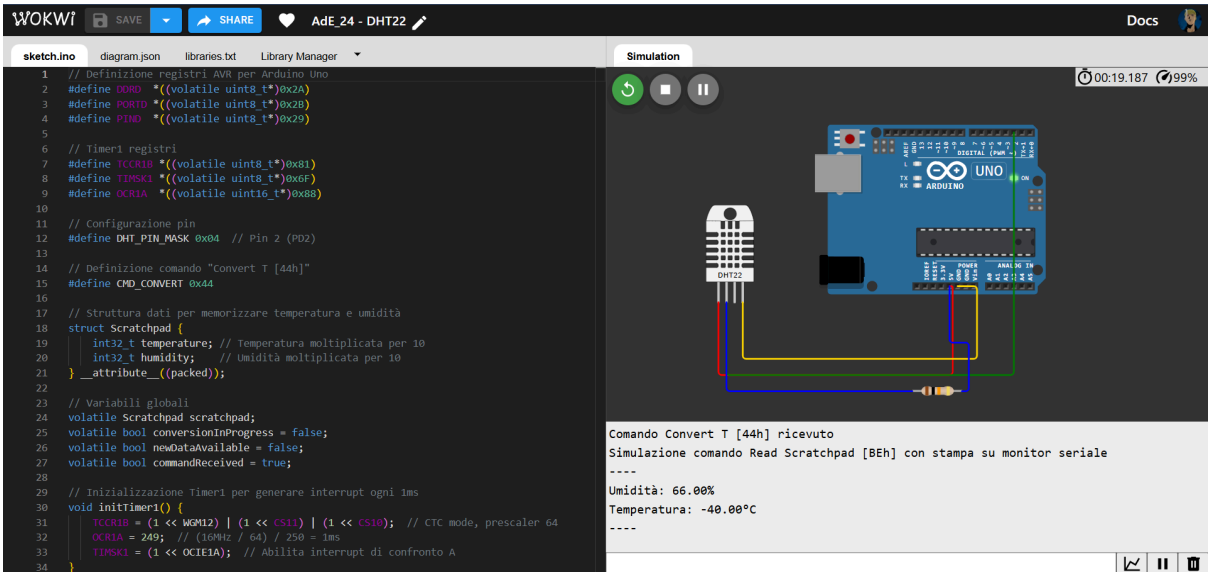


Immagine: risultato simulazione su Wokwi [rif.]

Il funzionamento del progetto è impostato in modo tale che, se si volesse avere la lettura di dati diversi, si devono cambiare i valori di temperatura e umidità mentre la simulazione è ancora in esecuzione, quindi cliccando sul sensore vedremo una finestra che ci permette di modificare i valori.

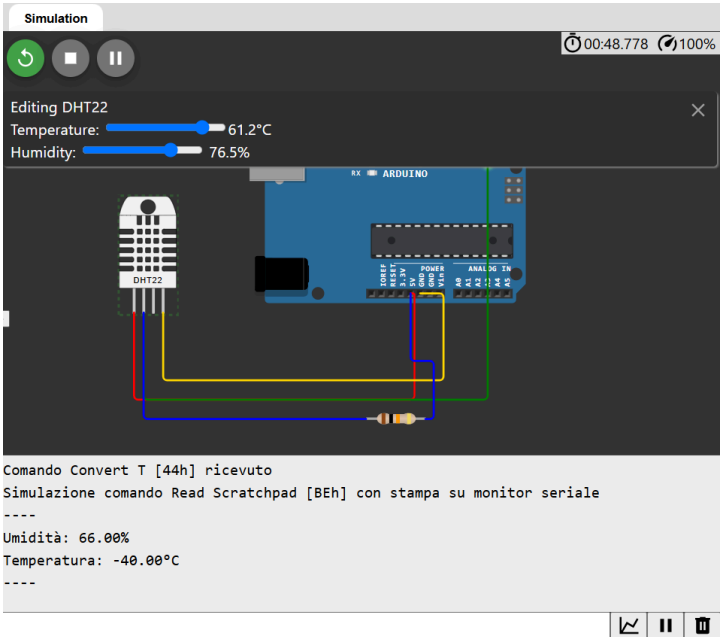


Immagine: impostazione valori da far leggere al sensore [rif.]

Per poi avere la lettura e conversione dei valori appena impostati bisogna riavviare la simulazione che stamperà nuovamente a schermo i valori appena impostati, simulando così la ricezione di un nuovo comando Convert T [44h].

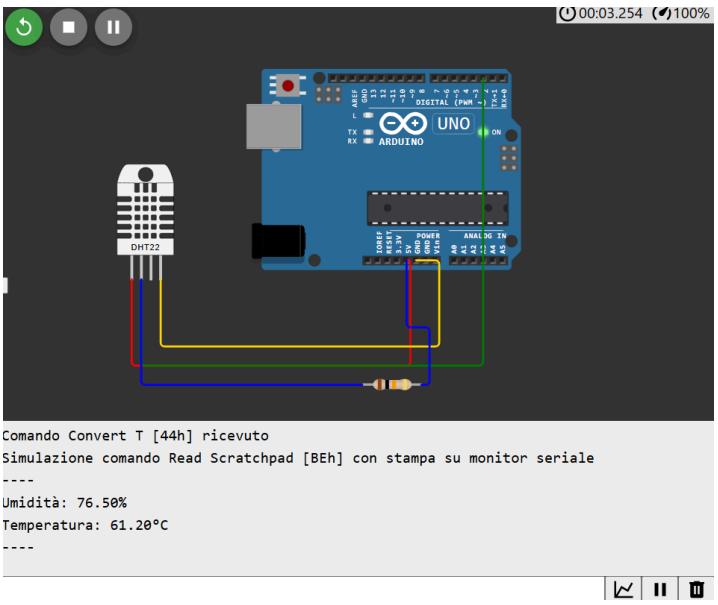


Immagine: nuova lettura con dati impostati precedentemente [rif.]

4. Bibliografia

[1] https://www.adrirobot.it/dht22-am2302-sensore-temperatura-umidita/
[2] https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf
[3] 1-Wire
[4] https://components101.com/microcontrollers/arduino-uno
[5] http://www.microcontroller.it/Tutorials/Elettronica/componenti/ds1820.htm